23MAT116 Discrete Mathematics
Exam Crash Guide

All likely question types and variations from the uploaded lecture slides
+ a Rosen-based relations section

Built for last-day revision

Use this as a question-pattern map, not as a worked-solution book.

Source map

- Lecture slides P1-P23 cover logic, propositions, equivalences, predicates, quantifiers, inference, proofs, counting, pigeonhole principle, recurrence relations, generating functions, and advanced counting.
- The Rosen textbook is used for the relations chapter: properties of relations, closures, equivalence relations, partitions, partial orders, Hasse diagrams, and topological sorting.

How to use this tonight

1. First identify the topic family: logic, quantifiers, proofs, counting, pigeonhole, recurrence, generating functions, or relations.
2. Then match the wording of the question to the template in the guide.
3. For any proof question, decide whether direct proof, contraposition, contradiction, or counterexample fits best.
4. For any counting question, decide whether it is a product-rule, sum-rule, complement, permutation, combination, inclusion-exclusion, or pigeonhole problem.
5. For any relation question, check the property being asked: reflexive, symmetric, antisymmetric, transitive, equivalence, or partial order.

1. Propositional logic and logical equivalences

Question types

- Decide whether a sentence is a proposition or not.
- Find the truth value of a simple proposition or of a compound proposition.
- Translate English into symbols, and symbols back into English.
- Build a truth table for a compound proposition.
- Decide whether a proposition is a tautology, contradiction, or contingency.
- Show that two propositions are logically equivalent.
- Find the negation of a proposition, especially when the sentence contains and, or, if...then, unless, only if, and iff.
- Check whether two English statements say the same thing.
- Detect a fallacy in an argument that looks logical but is invalid.

Common variations you may see

- If p then q; p only if q; p implies q; q whenever p; q if p; p is sufficient for q; q is necessary for p.
- p iff q; p is necessary and sufficient for q; if p then q, and conversely.
- Negations of 'and', 'or', 'either…or', 'neither…nor', 'unless', and 'not both'.
- Questions with variables left blank in a truth table.
- Equivalence questions where you must use De Morgan, distributive, associative, commutative, identity, domination, idempotent, absorption, double negation, or negation laws.
- Counterexample questions where you only need one assignment of truth values to break an equivalence.

The translation patterns to remember

English phrase	Symbolic meaning
if p then q	$p \to q$
p only if q	$p \to q$
q if p	$p \to q$
q whenever p	$p \to q$
p is sufficient for q	$p \to q$
q is necessary for p	$p \to q$
p iff q	$(p \to q) \land (q \to p)$
p unless r	$\neg r \to p$, or $p \lor r$ depending on wording
either p or q, but not both	$(p \lor q) \land \neg(p \land q)$

What exam questions usually ask you to do

- Show that a statement is equivalent to a simpler one, often by a chain of equivalences.
- Use a truth table with 2, 4, or 8 rows depending on the number of variables.
- Identify the meaning of an English sentence about campus, grades, internet access, or classes.
- Negate a statement cleanly without leaving a negation in front of the whole sentence.

2. Predicate logic and quantifiers

Question types

- Decide what a predicate is and what makes it true or false.
- Write a proposition from a predicate once values are substituted.
- Translate English statements with words like all, every, some, there exists, none, not all, and exactly one.
- Negate quantified statements correctly.
- Work with nested quantifiers and pay attention to the order.
- Switch between set-builder notation and quantified statements.
- Read and write truth sets.

Common variations you may see

- Domain is all people, integers, real numbers, students in the class, or a specified finite set.
- Statements involving 'at least one', 'exactly one', 'at most one', 'no one', or 'everyone'.
- A pair or triple predicate such as Q(x, y).
- Questions asking for the truth value of P(a), Q(a, b), or a quantified claim over a small domain.
- Negations of universal and existential statements in a form where no negation sits in front of a quantifier.

Quantifier negation template

Original statement	Negation
$\forall x\, P(x)$	$\exists x\, \neg P(x)$

$\exists x\, P(x)$	$\forall x\, \neg P(x)$
$\forall x(P(x) \rightarrow Q(x))$	$\exists x(P(x) \wedge \neg Q(x))$
$\neg \forall x\, P(x)$	$\exists x\, \neg P(x)$
$\neg \exists x\, P(x)$	$\forall x\, \neg P(x)$

A good exam habit: first write the predicate, then the domain, then the quantifier order, then negate carefully if needed.

3. Rules of inference and valid arguments

Question types

- Identify the rule of inference used in a short argument.
- Test whether an argument is valid or invalid.
- Build a conclusion from several premises involving if-then statements and universal or existential claims.
- Spot an error in existential instantiation or universal generalization.
- Turn an English argument into symbolic logic and then derive the conclusion.

Rules to know cold

- Modus ponens: $p \rightarrow q$, p, therefore q
- Modus tollens: $p \rightarrow q$, $\neg q$, therefore $\neg p$
- Hypothetical syllogism: $p \rightarrow q$, $q \rightarrow r$, therefore $p \rightarrow r$
- Disjunctive syllogism: $p \vee q$, $\neg p$, therefore q
- Addition: p, therefore $p \vee q$
- Simplification: $p \wedge q$, therefore p
- Conjunction: p, q, therefore $p \wedge q$
- Universal instantiation and universal generalization
- Existential instantiation and existential generalization

Typical traps

- Do not reuse the same constant for two different existential instantiations unless the premises really justify it.
- Do not generalize from one example unless the argument is valid for all objects.
- Do not treat 'if p then q' as 'if q then p'.
- Do not confuse 'some' with 'all'.

4. Proof methods

Question types

- Prove a divisibility statement.
- Prove a parity statement about odd and even integers.
- Prove an inequality or an existence statement.
- Prove a statement by contraposition or contradiction.
- Provide a counterexample to show a statement is false.
- Use a proof by cases when the statement splits naturally.

The proof method you should pick

- Direct proof: best for implications where definitions can be unpacked immediately.
- Contraposition: best when the conclusion is a negation or when the contrapositive is easier to start from.

- Contradiction: best when the statement says 'there is no…' or 'there exists no…' or when the desired conclusion is hard to reach directly.
- Counterexample: best when the statement claims something is always true but is actually false.

Common proof families from the slides and textbook-style exercises

- If n is even, then 3n + 4 is even.
- If 5n is odd, then n is odd.
- Every prime greater than 3 is one more or one less than a multiple of 6.
- The sum of three consecutive integers is divisible by 3.
- There is no smallest positive rational number.
- If a and b are divisible by 5, then a + b is divisible by 5.
- If n is divisible by 4, then n squared is divisible by 4.
- If a is divisible by 6 and b is divisible by 6, then a - b is divisible by 6.

What the proof usually starts with

- Write the integer in the form k = 2m, 2m + 1, 6m, 6m ± 1, or similar.
- Use the definition of divisibility.
- Use the definition of odd and even.
- Use algebraic rearrangement to expose a multiple of the target number.

5. Counting: sum rule, product rule, permutations, combinations

Question types

- Count the number of strings meeting one or more restrictions.
- Count the number of ways to choose committees, teams, rooms, passwords, or phone numbers.
- Count functions from one finite set to another.
- Count arrangements with or without repetition.
- Count using complementary counting or inclusion-exclusion.

Core formulas to remember

Situation	Count
Product rule	Multiply the number of choices at each step
Sum rule	Add counts for disjoint cases
Number of functions from A to B	|B|^|A|
Permutations of n distinct objects	n!
r-permutations from n	n!/(n-r)!
Combinations	nCr = n!/(r!(n-r)!)
With repetition of letters	count each position independently
Without repetition	subtract used choices each step

Common variations you may see

- Eight-letter strings with no vowels / at least one vowel / exactly one vowel / starting with X / ending with X / repeated letters allowed or not.
- Three-digit numbers with digit restrictions, such as exactly two 4s or first digit odd.
- Committee or team selection from multiple groups.
- Passwords with length restrictions and at least one digit.
- Counting subsets with a condition like 'more than one element'.
- Counting palindromes and bit strings with structural constraints.

How to attack a counting problem

- Break the object into positions or stages.
- Decide whether order matters.
- Decide whether repetition is allowed.
- Use a complement if the restriction is easier to subtract than to count directly.
- If the problem says 'at least one' or 'at least one of two conditions', inclusion-exclusion is often the right tool.

6. Pigeonhole principle and guaranteed existence

Question types

- Find the minimum number of objects needed to guarantee a repeat.
- Show that two selected objects must share a property.
- Show that some pair or subset must satisfy a condition.
- Use the generalized pigeonhole principle.
- Prove a statement by thinking in terms of boxes, buckets, or categories.

Template

If n objects are placed into k boxes, then some box contains at least ceil(n/k) objects.

Typical variations

- Birthday/month/grade/suit/letter/class/color problems.
- Cards drawn from a deck, socks from a drawer, students with grades, or houses with numbers.
- Two different days with the same pizza/topping combination.
- Consecutive numbers or equal remainders.
- Geometric pigeonhole problems, such as points inside a triangle.
- Monotone subsequence problems and Ramsey-type questions.

What to do on these questions

- Define the pigeons clearly.
- Define the holes clearly.
- Count the holes first, then add one extra pigeon to force a collision.
- For stronger problems, use a smarter pigeonhole choice such as residue classes, months, suits, or intervals.

7. Recurrence relations and generating functions

Question types

- Write a recurrence relation from a counting process.
- Classify a recurrence as linear, homogeneous, constant-coefficient, or not.
- Solve a linear homogeneous recurrence relation.
- Find a closed form from initial conditions.
- Use generating functions to encode and solve a recurrence.
- Work with divide-and-conquer style recurrences.

Common recurring models

- Fibonacci-style rabbit growth.
- Tower of Hanoi.
- Bit strings with forbidden substrings.

- Ordered sums with restrictions.
- Sequences where each term depends on one or more previous terms.

What to check first

- Is the recurrence linear?
- Is it homogeneous?
- Are the coefficients constant?
- What is the degree / order of the recurrence?
- What are the initial conditions?
- Can it be solved by characteristic roots or by generating functions?

Generate function questions can ask you to do

- Turn a recurrence into a formal power series.
- Multiply, shift, and isolate the generating function.
- Extract coefficients from the final closed form.
- Use generating functions to prove a combinatorial identity.

8. Relations (from Rosen chapter 9)

What the examiner may ask

- Define a binary relation, relation on a set, and n-ary relation.
- Represent a relation as a set of ordered pairs, matrix, or digraph.
- Check whether a relation is reflexive, symmetric, antisymmetric, or transitive.
- Find the reflexive, symmetric, or transitive closure.
- Find the inverse relation and composition of relations.
- Find equivalence classes and partitions.
- Work with partial orders and Hasse diagrams.
- Find maximal, minimal, greatest, least, upper bound, lower bound, lub, glb.
- Give a compatible total order using topological sorting.

Relation properties quick check

Property	Check
Reflexive	(a, a) is in R for every a
Symmetric	If (a, b) is in R then (b, a) is in R
Antisymmetric	If (a, b) and (b, a) are in R, then a = b
Transitive	If (a, b) and (b, c) are in R, then (a, c) is in R

Closures and what they mean

- Reflexive closure: add all missing diagonal pairs (a, a).
- Symmetric closure: add all reverse pairs (b, a).
- Transitive closure: add all pairs forced by paths; this is the connectivity relation.
- In matrix form, closures are often tested with zero-one matrices or Warshall-style reasoning.

Equivalence relations and partitions

- An equivalence relation is reflexive, symmetric, and transitive.
- The equivalence class of an element is the set of all elements related to it.
- Equivalence classes form a partition.
- Given a partition, you can build an equivalence relation by relating elements in the same block.

- Congruence modulo m is a standard example.

Partial orders and Hasse diagrams

- A partial order is reflexive, antisymmetric, and transitive.
- Common examples: divisibility, subset inclusion, task precedence.
- A Hasse diagram removes loops and transitive edges and draws edges upward.
- Topological sorting gives a compatible total order.
- Questions often ask for maximal/minimal elements, greatest/least elements, bounds, and lattice properties.

n-ary relations and database-style questions

- Primary key and composite key ideas.
- Projection: delete fields from a relation.
- Join: combine relations that agree on common fields.
- These can show up as conceptual questions rather than pure computation.

Typical relation exam prompts

- Is this relation an equivalence relation? If not, which property fails?
- Find the equivalence classes of a congruence relation or a custom relation.
- Draw the Hasse diagram for a relation like divisibility or subset inclusion.
- Find all compatible total orderings or one possible topological sort.
- Use a matrix or digraph to decide whether a relation has a given property.

9. Ultra-brief graph theory checklist

If graph theory is included in your exam, be ready for these question shapes:

- Define graph, subgraph, degree, walk, path, connected graph, and shortest path.
- Identify Euler graphs / Euler trails and Hamilton cycles.
- Use Fleury's algorithm at a basic level.
- Recognize shortest-path style questions and simple graph representations.

10. One-minute decision tree for any question

6. Is it asking to translate language? Then use propositional logic or predicate logic.
7. Is it asking to prove something? Then check if direct, contraposition, contradiction, or counterexample fits.
8. Is it asking to count objects? Then choose product rule, sum rule, permutations, combinations, inclusion-exclusion, or complement.
9. Is it asking for 'at least one repeated' or 'must exist'? Then think pigeonhole.
10. Is it about sequences or repeated structure? Then think recurrence or generating functions.
11. Is it about ordered pairs, matrices, partitions, or Hasse diagrams? Then think relations.

11. Final formula sheet

- $p \rightarrow q \equiv \neg p \vee q$
- $\neg(p \wedge q) \equiv \neg p \vee \neg q$
- $\neg(p \vee q) \equiv \neg p \wedge \neg q$
- $\neg \forall x\, P(x) \equiv \exists x\, \neg P(x)$
- $\neg \exists x\, P(x) \equiv \forall x\, \neg P(x)$
- Number of functions from a set of size m to a set of size n: n^m

- Generalized pigeonhole: some box has at least ceil(n/k) objects
- Equivalence relation = reflexive + symmetric + transitive
- Partial order = reflexive + antisymmetric + transitive

Now upload this to your phone and revise by question type, not by slide order.